

Evaluating HebbSPRT as a Memory Layer for Continual Learning in Digit Classification

Introduction

Continual learning studies the setting in which a model learns a sequence of tasks rather than receiving all classes at once. A useful model should learn new tasks while retaining useful knowledge from previous tasks. Standard neural networks often fail in this setting because training on later tasks can overwrite parameters that were important for earlier tasks. This failure mode is usually called catastrophic forgetting.

This project evaluates whether HebbSPRT can be used as an intermediate memory layer inside a multilayer perceptron (MLP) to reduce forgetting. The main idea is to let a Hebbian/Sanger-style rule learn stable feature filters, and then use a sequential probability ratio test (SPRT) to decide which filters have converged. Once a filter is judged to be stable, it is frozen and no longer updated by the memory rule.

The experiment uses the scikit-learn digits dataset. Each input is an 8 x 8 grayscale image, flattened into a 64-dimensional vector. To simulate class-incremental continual learning, the ten digit classes were divided into five binary tasks:

Task	Classes
Task 1	0 / 1
Task 2	2 / 3
Task 3	4 / 5
Task 4	6 / 7
Task 5	8 / 9

The experiment uses a single-head class-incremental setting. The model always has one ten-class output head, is not given a task identifier at test time, and does not use a replay buffer. Each task is trained for 100 epochs. After completing each new task, the model is evaluated on all tasks that have already been seen.

Methods

Two models were compared. The first model was a standard supervised MLP baseline:

Plain MLP: 64 -> 256 -> 256 -> 128 -> 10

The second model inserted a HebbSPRT memory branch after the first hidden layer:

HebbMemoryMLP: 64 -> 256 -> HebbSPRT(256, 64) -> concat -> 256 -> 128 -> 10

Let h denote the 256-dimensional hidden representation produced by the first fully connected layer. HebbSPRT maintains a weight matrix W with shape 64 x 256. Each row of W is a feature filter, or equivalently a direction in the hidden representation space. The memory response vector is computed as:

$$v = h W^T$$

Thus, v measures how strongly the current hidden representation activates each learned filter. The supervised part of the network receives the concatenation of the original hidden representation and the memory response. Conceptually, the model has access to both the current MLP feature h and the HebbSPRT memory feature v .

The HebbSPRT update has two main components. First, a Sanger-style Hebbian update is used so that later filters learn residual structure that has not already been explained by earlier filters. For filter k , omitting the batch index for readability, the update can be written approximately as:

$$r_k = h - \sum_{i \leq k} v_i w_i$$

$$\Delta w_k = \eta \text{mean}(v_k r_k)$$

Only filters that have not yet been frozen are updated. Second, SPRT is used as a freezing rule. After each update, the algorithm records the movement size of each filter using an exponentially smoothed drift statistic:

$$d_k \leftarrow 0.9 d_k + 0.1 \|\Delta w_k\|_2$$

Small drift means that the filter is no longer moving much. The SPRT mechanism compares evidence for a low-drift, converged state against evidence for a higher-drift, still-learning state. In this implementation, the low and high drift references were 0.006 and 0.020, respectively. When the log-likelihood-ratio evidence for a filter exceeds the decision threshold A , that filter is marked as frozen.

Evaluation Metrics

Continual learning performance cannot be evaluated only by the final task accuracy, because a model can perform well on the newest task while forgetting all earlier tasks. This project therefore uses two main metrics.

The first metric is the accuracy matrix. Let $A[t, j]$ denote the accuracy on task j after the model has finished training task t :

$$A[t, j] = \text{accuracy after training task } t, \text{ evaluated on task } j$$

The diagonal entries show immediate performance on the task that was just learned. Entries to the left of the diagonal show how much old-task performance remains after later tasks are learned.

The second metric is forgetting. For each old task j , forgetting is defined as the difference between the best accuracy ever achieved on that task and the final accuracy on that task:

$$F_j = \max_t A[t, j] - A[T, j]$$

Lower forgetting indicates better retention of previous tasks. Final average accuracy across all five tasks is also reported.

Results and Discussion

The accuracy matrices show that both models learned the currently active task very well, but both models lost nearly all performance on previous tasks after training moved to later tasks. The PlainDeepMLP achieved immediate accuracy values of 1.00 on tasks 0/1, 2/3, 4/5, and 6/7, and 0.96 on the final task 8/9. The HebbMemoryMLP also achieved strong immediate task performance, with diagonal values of 1.00, 1.00, 0.98, 1.00, and 0.98.

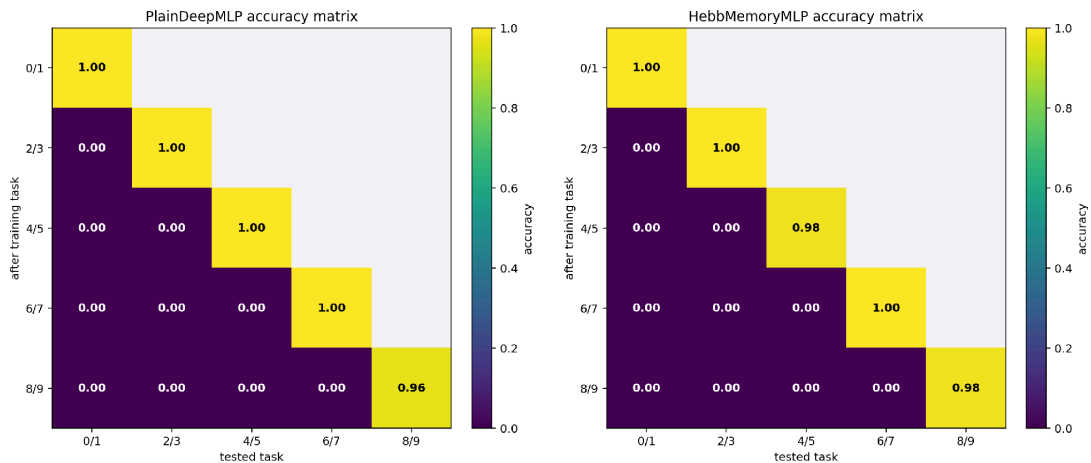


Figure 1. Accuracy matrices for the baseline MLP and the HebbSPRT-memory MLP.

However, the off-diagonal entries are the key evidence for continual learning. After later tasks were trained, the accuracies on earlier tasks dropped to 0.00 for both models. At the end of the full sequence, the PlainDeepMLP retained only the final task, with final average accuracy 0.1925. The HebbMemoryMLP also retained only the final task, but its final task accuracy was slightly higher, producing final average accuracy 0.1962.

Model	Final avg accuracy	Average forgetting
PlainDeepMLP	0.1925	1.0000
HebbMemoryMLP	0.1962	0.9954

The average accuracy over seen tasks declined almost identically for the two models. This pattern means that HebbSPRT did not produce a meaningful improvement in retention under the current training setup.

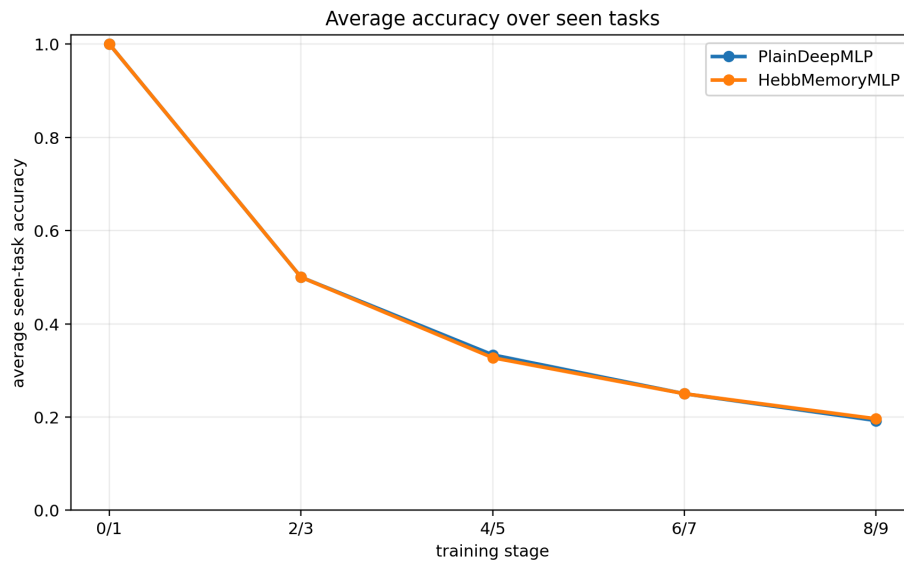


Figure 2. Average accuracy over all tasks seen so far during sequential training.

The forgetting metric gives the same conclusion. The average forgetting of the PlainDeepMLP was 1.0000, while the average forgetting of the HebbMemoryMLP was 0.9954. The Hebb model was numerically slightly better, but the difference was too small to support the claim that the memory layer solved catastrophic forgetting.

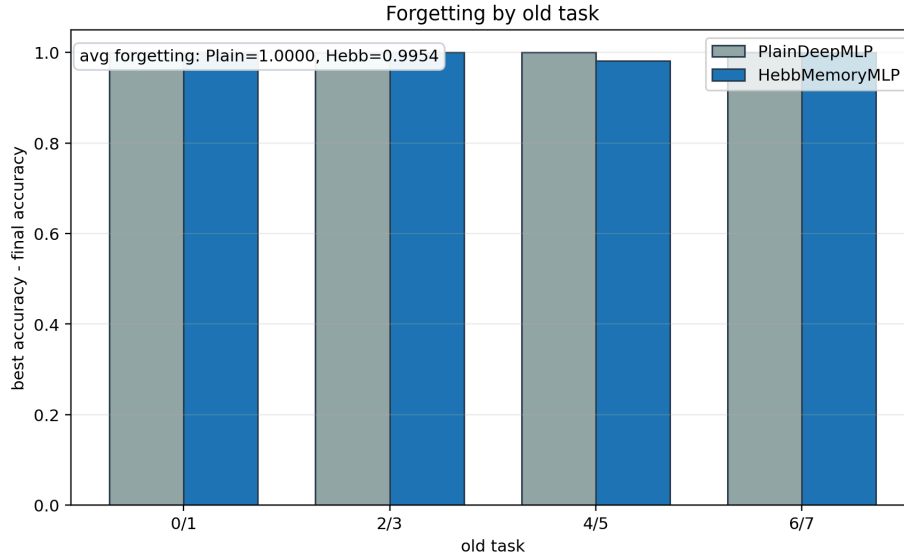


Figure 3. Forgetting by old task for the baseline and HebbSPRT-memory models.

The internal HebbSPRT diagnostics show that the memory mechanism itself was active. The number of frozen filters increased from 0 to 64 across training, the mean drift fell close to zero, and the LLR evidence rose far above the decision threshold. These patterns indicate that filters were being judged as stable and then frozen by the SPRT rule.

Conclusion

This experiment shows that HebbSPRT can be embedded into an MLP and that its internal filter-freezing mechanism works as designed. The filters converge, drift decreases, and SPRT evidence triggers freezing. However, under the current single-head, no-replay continual learning setting, this mechanism is not sufficient to prevent catastrophic forgetting.

The final result is therefore a negative but informative result. PlainDeepMLP had average forgetting of 1.0000, while HebbMemoryMLP had average forgetting of 0.9954. The HebbSPRT version was only marginally better. A more precise conclusion is that feature freezing alone is not enough; old-task decision boundaries also need protection. Future extensions could include a small replay buffer, constraints on old-class output weights, or a design in which frozen Hebb features are used more directly for old-class classification.

Resources

1. Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*. 2011;12:2825-2830.
2. McCloskey M, Cohen NJ. Catastrophic interference in connectionist networks: The sequential learning problem. In: *Psychology of Learning and Motivation*. 1989;24:109-165.
3. Oja E. Simplified neuron model as a principal component analyzer. *Journal of Mathematical Biology*. 1982;15:267-273.
4. Sanger TD. Optimal unsupervised learning in a single-layer linear feedforward neural network. *Neural Networks*. 1989;2(6):459-473.
5. Wald A. Sequential Tests of Statistical Hypotheses. *Annals of Mathematical Statistics*. 1945;16(2):117-186.